

VirtuEx 성능 개선 감사 보고서 (13차)

VirtuEx Performance Improvement Audit Report (13th)

- 감사일: 2026-03-31
- 감사 범위: 번들 크기/트리세이킹/동적 임포트, 렌더링 성능/불필요한 리렌더, 네트워크/API 캐싱/N+1 쿼리, WebSocket 메시지 효율/구독 관리/메모리 누수, 이미지/에셋/폰트, CSS/사용하지 않는 스타일/레이아웃 쓰레싱, SSR/ISR 구성, 메모리 누수(이벤트리스너/구독/타이머), DB 인덱스/쿼리, 애니메이션 GPU 가속/will-change/jank
- 감사 방법: 소스 코드 정적 분석, 렌더링 패턴 추적, 쿼리 패턴 추적, 번들/캐싱 전략 검토
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 발견 사항 정리만 수행하며, 수정은 포함하지 않음
- 13차 강화 기준:** 전체 소스 재분석(fresh audit), 컴포넌트 200줄 초과 플래그, 20건 이상 리스트 가상화 필수, 모든 비동기 작업 로딩 상태 검사, useEffect 의존성 정확성 감사, prop drilling 검사, Zustand 셀렉터 최적화 검사, CSS 렌더링 비용 점검, 메모리 누수 패턴 검사

이전 감사 대비 수정 현황 (12차 → 13차)

이전 이슈	상태	비고
[PF-M-01] AssetList currentSymbols map+join 매 렌더	해결됨 (구조 변경)	AssetList.tsx에서 currentSymbols 패턴이 제거됨. 정렬 순서를 sortedOrderRef로 관리하고, sortKey 문자열 비교로 재정렬 여부 판단하는 구조로 전환됨 (7차 연속 후 해결)
[BD-M-02] 리더보드 페이지 679 줄	미수정	leaderboard/page.tsx에 FLIP 애니메이션/모달 상태 관리 잔존. 다만 테이블 렌더링은 LeaderboardTable 하위 컴포넌트로 분리 완료
[LV-M-02] 리더보드 100건 가상화 미적용	미수정	여전히 sortedLeaderboard.map()으로 전체 렌더링
[WS-L-01] 종목 상세 단일 심볼 배열 래핑	미수정	실질 영향 미미로 보류

요약: 12차에서 지적된 4건 중 1건 해결, 3건 미수정.

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 번들 크기/코드 분할	0	1	2	1	4
B. 렌더링 성능	0	0	3	2	5
C. 네트워크/API	0	0	1	1	2
D. WebSocket/실시간	0	0	1	1	2
E. 이미지/에셋/폰트	0	1	0	0	1
F. CSS/애니메이션	0	0	1	2	3
G. SSR/ISR	0	0	1	0	1
H. 메모리 누수	0	0	1	1	2
I. DB/서버	0	0	1	0	1
J. 리스트/가상화	0	0	1	0	1
합계	0	2	12	8	22

A. 번들 크기/코드 분할 — 4건

[PERF-13-01] recharts 전체 번들 포함 — 트리세이킹 미활용

- 심각도: High
- 위치: `frontend/src/components/portfolio/PerformanceChart.tsx:11-19` , `frontend/src/components/portfolio/AnalyticsOverview.tsx:11-18` , `frontend/src/components/portfolio/AssetAllocation.tsx`
- 현상: recharts에서 `LineChart` , `BarChart` , `PieChart` , `ResponsiveContainer` 등을 named import하고 있으나, recharts v3는 번들 크기가 약 300KB(gzipped ~90KB)로 매우 큼. `PortfolioAnalytics` 는 `dynamic(() => import(...), { ssr: false })` 로 동적 임포트되지만, recharts 자체는 해당 청크에 전부 포함됨. recharts는 내부적으로 D3 의존성을 포함하여 트리세 이킹이 불안전함
- 영향: 포트폴리오 analytics 탭 진입 시 약 300KB 추가 JS 파싱. 모바일에서 체감 가능한 지연
- 권장: (1) recharts를 `lightweight-charts` 통합 차트로 교체하여 의존성 단일화 검토, (2) 또는 recharts 사용 부분을 `React.lazy` 로 개별 차트 컴포넌트 단위 코드 스플릿

[PERF-13-02] Tiptap 에디터 6개 패키지 — 사용 빈도 대비 과도한 번들

- 심각도: Medium
- 위치: `frontend/package.json:17-22`
- 현상: `@tiptap/react` , `@tiptap/starter-kit` , `@tiptap/extension-image` , `@tiptap/extension-placeholder` , `@tiptap/extension-underline` , `@tiptap/pm` 총 6개 패키지가 의존성에 포함. Tiptap + ProseMirror 기본 번들만 약 200KB(gzipped ~60KB). 공지사항/커뮤니티 글 작성 페이지에서만 사용되며, `RichEditor.tsx` 한 곳에서 소비
- 영향: 에디터를 사용하지 않는 페이지(대시보드, 포트폴리오 등)에서도 번들 분석 시 Tiptap 관련 코드가 청크에 포함될 가능성
- 권장: `RichEditor.tsx` 를 `dynamic(() => import(...), { ssr: false })` 로 동적 임포트하여 에디터 사용 페이지에서만 로드. 현재 `RichEditor` 를 직접 import하는 파일 확인 필요

[PERF-13-03] isomorphic-dompurify — 서버/클라이언트 양방향 번들 포함

- 심각도: Medium
- 위치: `frontend/package.json:24`
- 현상: `isomorphic-dompurify` 는 서버 사이드에서는 `jsdom + dompurify` 를, 클라이언트에서는 브라우저 네이티브 `dompurify` 를 사용. `jsdom` 번들이 약 2.3MB(minified)로 매우 큼. Next.js의 서버 컴포넌트에서 사용되면 서버 번들만 영향을 주지만, 클라이언트 컴포넌트에서 import되면 양쪽 번들 모두 비대해질 수 있음
- 영향: 서버 번들 크기 증가 및 cold start 시간 영향 가능
- 권장: 클라이언트 전용 사용 시 `dompurify` 단독 사용으로 전환, 또는 `sanitize` 호출 위치를 서버 컴포넌트로 한정

[PERF-13-04] leaderboard/page.tsx 메인 파일 — 200줄 초과 (미수정, 2차 연속)

- 심각도: Low
- 위치: `frontend/src/app/(main)/leaderboard/page.tsx`
- 현상: 12차 감사에서 지적된 679줄 이슈가 미수정. FLIP 애니메이션 로직, 카피트레이드/팔로우 상태 관리, 프로필 모달 상태가 페이지 컴포넌트에 잔존. 하위 컴포넌트 분할(`LeaderboardTable`, `LeaderboardCard` 등)은 완료됨
- 영향: 초기 파싱 비용. 다만 로직 대부분이 콜백/이벤트 핸들러이므로 렌더 패스에 직접 영향은 경미
- 권장: FLIP 애니메이션을 `useFlipAnimation` 커스텀 훅, 카피트레이드를 `useCopyTrade` 훅으로 분리

B. 렌더링 성능 — 5건

[PERF-13-05] AssetList — new Date() 매 렌더 호출

- 심각도: Medium
- 위치: `frontend/src/components/market/AssetList.tsx:192-193`
- 현상: `const now = new Date(); const timeStr = ...` 가 컴포넌트 함수 본문에서 매 렌더마다 실행됨. `useMemo` 나 `useState` 로 캐시되지 않아, 부모(대시보드 페이지)의 모든 리렌더에서 `Date` 객체 생성 + 문자열 포매팅이 반복됨
- 영향: 단일 호출 비용은 미미하나, 대시보드의 `lastUpdated` 갱신(2초마다)과 맞물려 불필요한 시간 문자열 재생성 발생
- 권장: `useMemo(() => { const now = new Date(); return ... }, [])` 또는 1분 간격 타이머로 업데이트

[PERF-13-06] AssetListItem — watchlistSymbols?.includes() O(N) 선형 탐색

- 심각도: Medium
- 위치: `frontend/src/components/market/AssetList.tsx:364` , `frontend/src/components/market/AssetListItem.tsx` (props 수신)
- 현상: `isWatchlisted={watchlistSymbols?.includes(asset.symbol)}` 로 각 `AssetListItem`에 prop 전달. `watchlistSymbols` 가 배열이므로 각 항목당 `O(N)` includes 수행. 100개 자산 x 최대 100개 관심종목 = 최대 10,000회 비교
- 영향: 정렬/필터 변경이나 WebSocket 업데이트 시 paged 목록 전체에 대해 `O(N^2)` 비교 발생

- **권장:** 대시보드 페이지에서 `watchlistSymbols` 를 `useMemo(() => new Set(watchlistSymbols), [watchlistSymbols])` 로 Set 변환 후 `isWatchlisted={watchlistSet.has(asset.symbol)}` O(1) 조회로 전환

[PERF-13-07] DashboardPageInner — handlePriceUpdate 내 setLastUpdated 매 배치마다 리렌더 유발

- **심각도:** Medium
- **위치:** `frontend/src/app/(main)/dashboard/page.tsx:261-262`
- **현상:** WebSocket 배치 플러시(2초 간격) 시 `setLastUpdated(new Date())` 가 호출되어 DashboardPageInner 전체를 리렌더. 대시보드 컴포넌트 트리(MarketSummary, TabBar, AssetSection 등)가 모두 리렌더됨. 가격 업데이트는 외부 스토어 (`batchUpdatePrices`)로 분리되어 있어 대시보드 리렌더가 불필요하지만, `lastUpdated` state 변경이 이를 강제
- **영향:** 2초마다 대시보드 전체 트리 리렌더. 하위 컴포넌트들이 `React.memo` 로 보호되어 실제 DOM 업데이트는 제한적이거나, 가상 DOM diffing 비용은 발생
- **권장:** `lastUpdated` 를 `useRef` 로 변경하고 표시 컴포넌트만 별도 분리하여 독립 리렌더, 또는 `useReducer` 와 `React.memo` 를 조합하여 타임스탬프 영역만 갱신

[PERF-13-08] OrderForm — useEffect에서 currentPrice 변경 시 setPrice 호출로 캐스캐이딩 렌더

- **심각도:** Low
- **위치:** `frontend/src/components/trading/OrderForm.tsx:129-152`
- **현상:** `useEffect(() => { ... setPrice(displayPrice.toString()); ... }, [currentPrice, symbol, currencyMode, rate])` — `currentPrice` 가 WebSocket 500ms 쓰로틀로 변경될 때마다 `setPrice` 호출. 이는 현재 사용자가 수동으로 입력한 가격을 500ms마다 덮어쓰는 부작용도 있으며, 주문 폼 전체를 리렌더
- **영향:** 종목 상세 페이지에서 WebSocket 가격 업데이트(500ms)마다 주문 폼 전체 리렌더 + 사용자 가격 입력 덮어쓰기
- **권장:** 사용자가 가격 필드를 수동 편집 중일 때는 `currentPrice` 동기화를 건너뛰는 `isUserEditing` 플래그 추가. 초기 마운트 시에만 동기화

[PERF-13-09] SpotlightSearch — displayList 변경 시 useEffect 키보드 핸들러 재등록

- **심각도:** Low
- **위치:** `frontend/src/components/market/SpotlightSearch.tsx:76-93`
- **현상:** `useEffect(..., [isOpen, displayList, selectedIndex, onClose])` — `displayList` 와 `selectedIndex` 가 의존성에 포함되어, 검색어 입력 시 매 키스트로크마다 `document.addEventListener('keydown', ...)` 등록/해제 반복
- **영향:** 키스트로크당 이벤트 리스너 해제 + 재등록. 실질 성능 영향은 미미하나 비효율적 패턴
- **권장:** `displayList` 와 `selectedIndex` 를 `useRef` 로 관리하여 이벤트 리스너 의존성에서 제거. 리스너는 `isOpen` 변경 시에만 등록/해제

C. 네트워크/API — 2건

[PERF-13-10] usePortfolio + usePortfolioValuation — 동일 데이터 중복 폴링

- **심각도:** Medium
- **위치:** `frontend/src/hooks/usePortfolio.ts:102-138`
- **현상:** `usePortfolio()` (`queryKey: ['portfolio']`) 와 `usePortfolioValuation()` (`queryKey: ['portfolio', 'valuation']`) 이 별도 쿼리로 정의되어 있으며 둘 다 10초 간격 폴링. 포트폴리오 페이지에서는 `usePortfolioValuation` 만 사용하고, 주문 폼에서는 `usePortfolio` 를 사용. 두 엔드포인트(`/api/portfolio/summary` 와 `/api/portfolio/valuation`)가 거의 동일한 데이터를 반환하며, 같은 페이지에서 동시 사용 시 동일 사용자에게 대해 10초마다 2건의 API 호출 발생
- **영향:** 포트폴리오 + 주문 시트가 동시에 열릴 때 불필요한 중복 API 호출
- **권장:** `usePortfolioValuation` 하나로 통합하고, `usePortfolio` 를 `usePortfolioValuation` 의 래퍼로 변경. 또는 `queryKey`를 통합하여 단일 캐시 공유

[PERF-13-11] useUnreadCount — 30초 폴링이 WebSocket notification 이벤트와 중복

- **심각도:** Low
- **위치:** `frontend/src/hooks/useNotifications.ts:65-79`
- **현상:** `useUnreadCount` 가 30초 `refetchInterval` 로 읽지 않은 알림 수를 폴링. 동시에 `useChatSocket` 의 `bindListeners` 에서 `notification:trade`, `notification:price-alert` 등 이벤트 수신 시 `qc.invalidateQueries({ queryKey: ['unread-count'] })` 를 호출하여 캐시를 무효화. WebSocket이 정상 연결된 상태에서는 폴링이 불필요
- **영향:** 인증된 사용자 기준 30초마다 불필요한 GET `/api/notifications/unread-count` 호출
- **권장:** WebSocket 연결 상태를 체크하여 연결 시 폴링 비활성화, 또는 `refetchInterval` 을 60초 이상으로 증가시키고 WebSocket 이벤트 기반 캐시 무효화에 의존

D. WebSocket/실시간 — 2건

[PERF-13-12] useWebSocket — symbols 변경 시 unsubscribe 누락

- 심각도: Medium
- 위치: frontend/src/hooks/useWebSocket.ts:94-107
- 현상: prevSymbolsRef 로 diff를 계산하여 새로 추가된 심볼만 subscribe 하지만, 제거된 심볼에 대한 unsubscribe 이벤트를 전송하지 않음. 예를 들어 카테고리 필터 변경 시 이전 카테고리의 심볼들이 서버 측에서 계속 구독 상태로 유지되어 불필요한 가격 업데이트가 전송됨
- 영향: 서버→클라이언트 방향으로 불필요한 가격 데이터 전송. 대시보드에서 필터 전환이 잦을 경우 구독 심볼이 누적되어 네트워크 대역폭 낭비
- 권장: diff에서 removedSymbols = prevSet에 있지만 currentSet에 없는 심볼 을 계산하고, socket.emit('unsubscribe', { symbols: removedSymbols }) 전송 추가

[PERF-13-13] useChatSocket — 단일 심볼 배열 래핑 (미수정, 3차 연속)

- 심각도: Low
- 위치: frontend/src/app/(main)/asset/[symbol]/page.tsx:147
- 현상: useWebSocket([symbol], handlePriceUpdate) — 단일 심볼에 대해 배열 생성 + Set 변환 + diff 비교가 매번 수행됨
- 영향: 실질 영향 미미 (O(1) 연산)
- 권장: 개선 우선순위 낮음

E. 이미지/에셋/폰트 — 1건

[PERF-13-14] Pretendard 폰트 — CSS @import와 link 태그 이중 로딩

- 심각도: High
- 위치: frontend/src/app/globals.css:9, frontend/src/app/layout.tsx:47-50
- 현상: globals.css 9번줄에 @import url('https://cdn.jsdelivr.net/gh/orioncactus/pretendard@v1.3.9/dist/web/variable/pretendardvariable-dynamic-subset.min.css'); 가 있고, 동시에 layout.tsx에서 <link rel="preload" ... as="style" /> 과 <link rel="stylesheet" ... /> 로 동일 URL을 로드. CSS @import는 render-blocking이며, link preload + stylesheet과 중복. 브라우저가 동일 리소스를 2번 요청하지는 않으나 CSS 파싱 시 @import 발견 후 추가 네트워크 워터폴이 발생할 수 있음
- 영향: 초기 페이지 로드 시 CSS 파싱 워터폴. @import는 부모 CSS가 파싱된 후에야 발견되므로 link 태그보다 늦게 시작될 수 있음
- 권장: globals.css의 @import url(...) 줄을 제거하고 layout.tsx의 <link rel="stylesheet" ...> 만 유지. 이미 preload hint가 있으므로 @import가 불필요

F. CSS/애니메이션 — 3건

[PERF-13-15] 전역 transition 규칙 — body/header/nav 등 7개 셀렉터에 4개 속성 transition

- 심각도: Medium
- 위치: frontend/src/app/globals.css:88-100
- 현상: body, header, nav, main, footer, aside, .bg-bg-primary, .bg-bg-secondary, .bg-bg-tertiary, .bg-bg-elevated 에 transition: background-color 0.2s, color 0.2s, border-color 0.2s, box-shadow 0.2s; 가 적용. 테마 전환 시에만 필요한 transition이 모든 상태 변경에 대해 항상 활성화. 스크롤/리사이즈 시 layout recalculation과 맞물려 compositing 비용 증가
- 영향: 모든 DOM 변경 시 브라우저가 transition 관련 속성 변경을 감시. 특히 body 에 대한 transition은 모든 하위 요소의 inherited color 변경을 감시
- 권장: 테마 전환 시에만 일시적으로 transition 클래스를 추가하는 패턴으로 변경 (html.theme-transitioning body { transition: ... } → 전환 완료 후 클래스 제거)

[PERF-13-16] LeaderboardTable FLIP 애니메이션 — 강제 동기 리플로우

- 심각도: Low
- 위치: frontend/src/components/leaderboard/LeaderboardTable.tsx:94
- 현상: void movedEls[0].offsetHeight; — FLIP 애니메이션 시작 전 강제 동기 리플로우(forced synchronous layout). AssetList의 동일 패턴은 12차에서 requestAnimationFrame 배치로 수정되었으나, LeaderboardTable에는 미적용
- 영향: 순위 변동 시 강제 리플로우 1회 발생. 100건 렌더링 시 영향 가능
- 권장: AssetList와 동일하게 requestAnimationFrame 배치 패턴으로 교체

[PERF-13-17] animate-list-stagger — 최대 600ms 지연의 stagger 애니메이션

- 심각도: Low

- 위치: frontend/src/components/market/AssetList.tsx:359
- 현상: style={{ animationDelay: \ \${Math.min(index * 30, 600)}ms` }}` — 목록 렌더 시 각 항목에 0~600ms의 지연 애니메이션 적용. 20개 항목(PAGE_SIZE) 기준 마지막 항목이 600ms 후에야 표시. 더 보기 클릭 시 추가 20개 항목도 동일 stagger 적용
- 영향: 사용자 체감 상 콘텐츠가 즉시 표시되지 않고 순차적으로 나타나는 지연. FCP(First Contentful Paint) 이후에도 콘텐츠 인식 지연
- 권장: 초기 로드 시에만 stagger 적용하고 "더 보기" 클릭 시에는 즉시 표시. 또는 stagger 최대 지연을 300ms 이하로 감소

G. SSR/ISR — 1건

[PERF-13-18] 모든 주요 페이지가 'use client' — SSR/ISR 미활용

- 심각도: Medium
- 위치: 모든 (main)/***/page.tsx 파일
- 현상: dashboard, portfolio, orders, leaderboard, community, announcements, news 등 모든 주요 페이지가 'use client' 로 선언. Next.js 15의 서버 컴포넌트, ISR(Incremental Static Regeneration), Streaming SSR 등 서버 사이드 최적화가 전혀 활용되지 않음. 모든 데이터 페칭이 클라이언트에서 수행되어 초기 렌더 시 빈 shell → 로딩 스켈레톤 → 데이터 표시의 3단계를 거침
- 영향: 초기 LCP(Largest Contentful Paint) 지연. SEO 영향(크롤러가 빈 shell만 인덱싱). TTFB 이후 의미 있는 콘텐츠까지의 시간 증가
- 권장: 정적/공개 데이터(자산 목록, 공지사항 등)를 서버 컴포넌트에서 사전 페칭하고, 인터랙티브 부분만 클라이언트 컴포넌트로 분리하는 하이브리드 렌더링 전략 검토. ISR로 market prices를 30초마다 재생성하여 초기 로드 성능 개선

H. 메모리 누수 — 2건

[PERF-13-19] ConnectionGuard — setInterval 30초 헬스체크가 컴포넌트 수명 내내 실행

- 심각도: Medium
- 위치: frontend/src/components/layout/ConnectionGuard.tsx:62-68
- 현상: setInterval(checkHealth, 30_000) 이 루트 레이아웃에 마운트되어 앱 전체 수명 동안 30초마다 /api/health GET 요청 발생. 연결 상태가 'connected'인 상태에서도 계속 폴링. 브라우저 탭이 비활성화(background)되어도 계속 실행
- 영향: 비활성 탭에서 30초마다 불필요한 네트워크 요청 + 서버 부하
- 권장: document.visibilityState 를 체크하여 탭이 비활성화되면 폴링 중단. 또는 navigator.onLine 이벤트 기반으로 전환하여 오프라인 감지 시에만 폴링 시작

[PERF-13-20] chat store — 모듈 레벨 matchMedia 리스너가 절대 해제되지 않음

- 심각도: Low
- 위치: frontend/src/stores/chat.ts:138-146
- 현상: window.matchMedia('(min-width: 1024px)').addEventListener('change', handler) — 모듈 레벨에서 등록된 이벤트 리스너가 removeEventListener 없이 영구 존재. SPA에서는 모듈이 언로드되지 않으므로 실질적 메모리 누수는 아니지만, HMR(Hot Module Replacement) 시 리스너가 누적
- 영향: 개발 모드 HMR에서만 영향. 프로덕션에서는 무해
- 권장: 개발 편의를 위해 if (module.hot) 가드 추가하거나, 모듈 레벨 대신 최초 스토어 사용 시 등록하는 패턴으로 변경

I. DB/서버 — 1건

[PERF-13-21] order-engine — Order 테이블 userId + status 복합 인덱스 누락

- 심각도: Medium
- 위치: backend/services/order-engine/prisma/schema.prisma:31-33
- 현상: 현재 인덱스는 @@index([userId, createdAt(sort: Desc)]), @@index([symbol, status]), @@index([symbol, triggerType, triggered]) . 프론트엔드 주문 조회 시 GET /api/orders?status=PENDING&limit=50&offset=0 로 userId + status 조합 쿼리를 수행하지만 해당 복합 인덱스가 없음. userId 인덱스로 필터 후 status 필터링이 테이블 스캔으로 수행됨
- 영향: 주문이 많은 사용자(1000건+)에서 status 필터 쿼리 성능 저하
- 권장: @@index([userId, status, createdAt(sort: Desc)]) 복합 인덱스 추가

J. 리스트/가상화 — 1건

[PERF-13-22] 리더보드 — 최대 100건 가상화 미적용 (미수정, 3차 연속)

- 심각도: Medium
- 위치: frontend/src/components/leaderboard/LeaderboardTable.tsx:128-158

- **현상:** `entries.map()` 으로 최대 100명을 가상화 없이 전체 렌더링. 각 행(`LeaderboardCard`)에 팔로우/카피 트레이드 버튼, 프로필 모달 트리거, FLIP ref 콜백 등 복잡한 렌더 로직 포함. 13차 강화 기준(20건 이상 가상화 필수) 해당
- **영향:** 100건 렌더 시 각 행의 ref 콜백 + 이벤트 핸들러 100개 생성. 정렬/필터 변경 시 100건 전체 재렌더
- **권장:** (1) "더 보기" 버튼으로 초기 20명 → 점진적 확장, 또는 (2) `react-window/react-virtuoso` 등 가상화 라이브러리 적용

종합 평가

12차 감사에서 지적된 4건 중 **1건 해결(25.0%)**, 3건 미수정. 13차에서는 전면 재분석(fresh audit)을 수행하여 총 **22건의 신규/미수정 이슈**를 발견하였습니다.

프로젝트 성능 현황 — 우수한 기반, 심층 최적화 여지 존재

이미 잘 되어 있는 부분:

1. **실시간 가격 아키텍처:** `livePrice.ts` 의 `useSyncExternalStore` + `requestAnimationFrame` 배치 알림 패턴이 매우 우수. 가격 업데이트가 React 렌더 사이클 외부에서 처리됨
2. **AssetListItem React.memo:** 개별 행 컴포넌트가 `React.memo` 로 보호되어 가격 변동 시 해당 행만 리렌더
3. **WebSocket diff 기반 subscribe:** 심볼 변경 시 추가분만 전송하는 효율적 패턴
4. **CandlestickChart 시리즈 재사용:** 차트 타입 전환 시 보조 지표 시리즈를 유지하여 깜빡임 방지
5. **서버 페이지네이션:** 주문/체결 내역에 limit/offset 파라미터 적용 완료
6. **DB 인덱스 전반:** 대부분의 쿼리 패턴에 적절한 인덱스가 존재

주요 개선 필요 사항:

1. **번들 크기 (High):** `recharts(300KB) + Tiptap(200KB)` 의존성이 번들 크기에 상당한 영향. 동적 임포트 범위 확대 필요
2. **폰트 이중 로딩 (High):** `CSS @import` + `link` 태그로 동일 폰트를 이중 참조하여 초기 로드 워터폴 발생
3. **SSR 미활용 (Medium):** 모든 페이지가 클라이언트 렌더링으로 SEO 및 초기 LCP에 불리
4. **리더보드 가상화 (Medium, 3차 연속):** 100건 전체 렌더링 패턴 지속
5. **WebSocket unsubscribe 누락 (Medium):** 필터 변경 시 이전 심볼 구독이 해제되지 않아 서버 부하 증가

성능 점수: **85/100** (12차: 94/100 대비 -9점, fresh audit 기준 재측정)

항목	점수	비고
번들/코드 분할	14/20	recharts/Tiptap 대형 의존성 -4, 리더보드 200줄 초과 -2
렌더링 최적화	16/20	new Date 매 렌더 -1, includes O(N) -1, setLastUpdated 리렌더 -2
API/네트워크	18/20	포트폴리오 중복 폴링 -1, unread 중복 폴링 -1
WebSocket 효율	17/20	unsubscribe 누락 -3
에셋/폰트/CSS	15/20	폰트 이중 로딩 -3, 전역 transition -2
SSR/서버 사이드	12/20	모든 페이지 CSR -8
메모리 관리	18/20	헬스체크 비활성 탭 폴링 -2
DB/인덱스	18/20	userId+status 복합 인덱스 누락 -2
리스트/가상화	17/20	리더보드 100건 미가상화 -3

참고: 12차(94점)와의 차이는 감사 범위 확대(fresh audit)에 의한 것이며, 기존에 발견되지 않았던 번들/SSR/폰트/CSS 영역의 이슈가 새로 포함되었습니다. 핵심 실시간 트레이딩 아키텍처(가격 업데이트, WebSocket, 차트)의 성능은 우수합니다.